Semana 21 — Conteúdo de 2 aulas

Computação em Nuvem e Virtualização

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v44

Semana 21 — Conteúdo de 2 aulas

- **Aula 33:** Revisão e Prática de Kubernetes
- **Aula 34:** Computação em nuvem para Big Data

Aula 33 — Revisão e Prática de Kubernetes

Objetivos da Aula

- Retomar os conceitos de **orquestração**.
- Praticar a criação de **objetos** no Kubernetes.
- Entender **escala, atualização e autocorreção** na prática.
- Consolidar a base antes de Big Data.

Revisão Rápida

- **Orquestração:** gerenciar contêineres em escala.
- **Control plane:** API Server, etcd, Scheduler, Controller Manager.
- **Nodes:** executam os Pods (kubelet + runtime).
- **Objetos:** Pod, Deployment, Service, ConfigMap/Secret.

Ambiente de Prática

- Opções: **minikube**, **kind** ou um cluster gerenciado.
- CLI **kubectl** para interagir com o cluster.
- Verificar: `kubectl cluster-info` e `kubectl get nodes`.
- Laboratório local é suficiente para aprender.

Criando um Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: { app: web }
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: web
          image: nginx:1.27
```

Aplicando e Observando

- `kubectl apply -f web.yaml`
- `kubectl get pods` — ver as réplicas.
- `kubectl get deployments` — estado do deployment.
- `kubectl describe pod <nome>` — detalhes e eventos.

Escalando

- `kubectl scale deployment web --replicas=5`
- Observar novos Pods sendo criados.
- **HPA:** escala automática por uso de CPU.
- Relação direta com **elasticidade** na nuvem.

Autocorreção (Self-Healing)

- `kubectl delete pod <nome>` em um dos Pods.
- O Deployment **recria** o Pod automaticamente.
- Demonstra a reconciliação do **estado desejado**.
- Base da **alta disponibilidade** em contêineres.

Expondo com Service

- `kubectl expose deployment web --port=80 --type=NodePort`
- O **Service** dá um endereço estável aos Pods.
- Tipos: ClusterIP, NodePort, LoadBalancer.
- Integração com balanceamento de carga na nuvem.

Rollout e Rollback

- `kubectl set image deployment/web nginx=nginx:1.26`
- Acompanhar com `kubectl rollout status`.
- Reverter: `kubectl rollout undo deployment/web`.
- **Zero downtime** com rolling updates.

Note

Kubernetes mantém o **estado desejado**: você declara; ele reconcilia. Essa é a essência da orquestração.

Resumo da Aula

- Praticamos **Deployment, Service, escala e rollout**.
- Vimos a **autocorreção** funcionando.
- Kubernetes entrega **elasticidade e disponibilidade**.
- Próxima aula: **Big Data na nuvem**.

Referências

- Kubernetes — *Documentação oficial*.
- Material das aulas 33 a 38.

Aula 34 — Computação em nuvem para Big Data

Objetivos da Aula

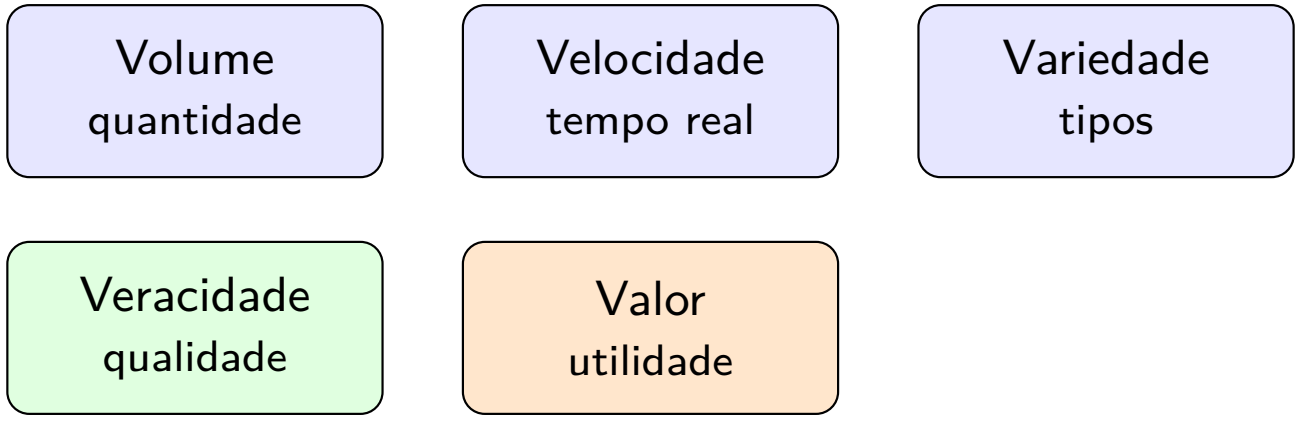
- Definir **Big Data** e seus desafios.
- Conhecer os **“V’s”** dos dados.
- Compreender por que a **nuvem** viabiliza Big Data.
- Identificar fontes e tipos de dados.

O que é Big Data?

- Conjuntos de dados **grandes e complexos** demais para ferramentas tradicionais.
- Exigem **armazenamento distribuído** e **processamento paralelo**.
- Presentes em redes sociais, sensores, logs, comércio.
- A nuvem fornece a **escala** necessária.

Os “V’s” do Big Data

- **Volume:** quantidade massiva de dados.
- **Velocidade:** geração e processamento em tempo real.
- **Variedade:** estruturados, semiestruturados e não estruturados.
- **Veracidade:** qualidade e confiabilidade.
- **Valor:** utilidade para o negócio.



Tipos de Dados

- **Estruturados:** tabelas com esquema definido.
- **Semiestruturados:** JSON, XML, logs.
- **Não estruturados:** texto, imagem, áudio, vídeo.
- A nuvem armazena todos com **armazenamento de objetos**.

Por que a Nuvem?

- **Escala elástica:** mais recursos sob demanda.
- **Armazenamento barato:** data lakes em objetos (ex.: S3).
- **Serviços gerenciados:** menos operação.
- **Integração** com analytics e machine learning.

Fontes de Dados

- **Web e redes sociais.**
- **Sensores e IoT.**
- **Logs de sistemas e aplicações.**
- **Transações** de comércio.
- **Dados abertos** governamentais.

Armazenamento Distribuído

- **HDFS:** sistema de arquivos distribuído do Hadoop.
- **Object storage:** S3, GCS, Azure Blob.
- **Data lake:** dados brutos centralizados.
- **Redundância** e tolerância a falhas.

Desafios

- **Coleta e limpeza** de dados.
- **Governança e privacidade** (LGPD).
- **Custo** de armazenar e processar.
- **Segurança** de dados sensíveis.

Note

Big Data na nuvem combina **armazenamento de objetos + processamento distribuído + serviços gerenciados** para escalar conforme a demanda.

Resumo da Aula

- **Big Data** é definido pelos “V’s”.
- A nuvem resolve **escala, custo e gestão**.
- Dados vêm de várias **fontes** e tipos.
- **HDFS e object storage** são bases de armazenamento.

Atividade Rápida

- Explique os **três V's** principais com exemplos.
- Dê exemplos de dados **estruturados x não estruturados**.
- Liste três motivos para usar a nuvem em Big Data.

Referências

- Apache Hadoop — *Documentação*.
- AWS — *Big Data*.
- BUYYA, R. et al. *Mastering Cloud Computing*.

Objetivos da Aula

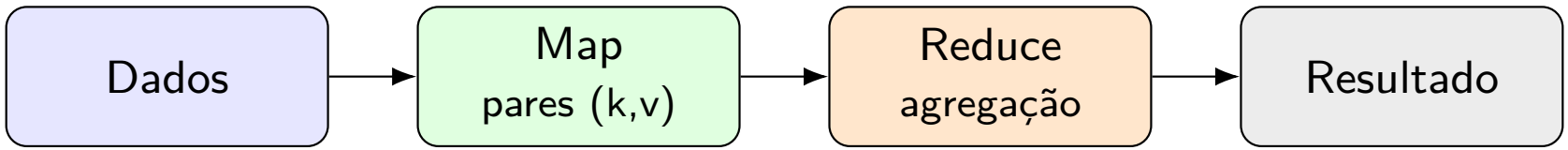
- Compreender o modelo **MapReduce** e o **Hadoop**.
- Conhecer **Spark** e o processamento em memória.
- Entender **pipelines de dados** e **ETL**.
- Identificar serviços gerenciados de Big Data.

Processamento Distribuído

- Dividir o trabalho em **muitas máquinas**.
- **Paralelismo** para processar dados em larga escala.
- Tolerância a falhas por **replicação**.
- Base: Hadoop, Spark e serviços gerenciados.

MapReduce

- Modelo de programação do Hadoop.
- **Map:** processa cada dado, gerando pares.
- **Reduce:** agrega os resultados.
- Divide-e-conquista em larga escala.



Hadoop

- Ecossistema de **armazenamento (HDFS)** e processamento (MapReduce).
- **YARN:** gerencia recursos do cluster.
- Amplamente usado em data lakes.
- Gerenciado na nuvem: **Amazon EMR**, Dataproc.

Apache Spark

- Processamento **em memória** — muito mais rápido.
- Suporta **batch, streaming, SQL e ML**.
- APIs em Python (**PySpark**), Scala, Java.
- Preferido para análises interativas e ML.

Pipelines e ETL

- **ETL:** Extração, Transformação e Carregamento.
- **ELT:** transformar dentro do destino (data warehouse).
- Ferramentas: AWS Glue, Azure Data Factory.
- Movem dados de fontes para análises.

Serviços Gerenciados

Necessidade	Serviço (exemplos)
Armazenamento	S3, GCS, Azure Blob
Processamento	EMR, Dataproc, Databricks
ETL	Glue, Data Factory
Streaming	Kinesis, Pub/Sub

Data Warehouse x Data Lake

- **Data lake:** dados brutos, flexível, barato.
- **Data warehouse:** estruturado, otimizado para consultas (Redshift, BigQuery).
- **Lakehouse:** combina ambos.
- Escolha conforme o **caso de uso**.

> **Note**
>
> **MapReduce** e **Spark** resolvem o mesmo problema de formas diferentes: Spark, em memória, tende a ser muito mais rápido para cargas interativas.

Resumo da Aula

- **MapReduce/Hadoop** introduziram o processamento distribuído.
- **Spark** acelera com processamento em memória.
- **ETL/ELT** movem e preparam os dados.
- Serviços **gerenciados** reduzem a operação na nuvem.

Atividade Rápida

- Explique as fases **Map** e **Reduce**.
- Cite vantagens do **Spark** sobre o MapReduce.
- Diferencie **data lake** de **data warehouse**.

Referências

- Apache Spark / Hadoop — *Documentação*.
- AWS EMR / Google Dataproc.